Restaurant App MVP — Software Requirements Specification

Fall 2026 | Frontend Only | Version 1.0

Client meeting / self-interview: As a customer, I would download the app if it lets me browse daily specials quickly, search the menu, see prices and availability, reserve a suitable table for a chosen time, add special instructions, apply a promo code, choose dine-in or takeaway, and track an order. I would also expect a simple profile and dark mode. These answers directly drive the requirements below.

1. Introduction

1.1 Purpose

This SRS defines the requirements for the Restaurant App MVP, a React Native frontend prototype for diners and restaurant managers. It describes the screens, local mock data, client-side state, hooks, navigation, UML model, and testable behaviour covered by the Fall 2026 assignment.

1.2 Scope

• In scope: login/signup validation, mock authentication, menu browsing, categories, search and debounce, sorting, favourites, cart, promo codes, order summary, table reservation, order tracking, customer profile/theme, manager dashboard, local persistence, and responsive React Native UI.

• Out of scope: real payments, real authentication servers, backend APIs, cloud database, push notifications, real restaurant inventory, external maps, external delivery services, and production security.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
MVP	Minimum Viable Product.
UML	Unified Modeling Language.
Hook	React function for state, effects, refs or reusable logic.
Context	React mechanism for shared state without prop drilling.
Reducer	Pure function that calculates the next state from an action.
Mock Data	Local sample data used without a backend.
FlatList	React Native list component optimized for scrolling data.
AsyncStorage	Local persistent key-value storage used by the prototype.

2. Overall Description

2.1 User Roles

Role	Permissions & Goals
Customer	Browse/search menu, manage cart, apply promo, reserve a table, place and track orders, manage profile/theme.
Restaurant Manager	View/update incoming order status and access reservation/menu management areas.

2.2 User Stories

• As a customer, I want to browse daily specials so I can choose quickly.

• As a customer, I want to search menu items so I can find a dish without scrolling.

• As a customer, I want to filter by category so I can focus on a type of food.

• As a customer, I want to see unavailable items disabled so I do not order unavailable food.

• As a customer, I want to reserve a table for a chosen time so I can arrive with a table ready.

• As a customer, I want to add special instructions so the kitchen receives my preferences.

• As a customer, I want to apply a promo code so I can see a discount before checkout.

• As a customer, I want to track my order so I know its current status.

• As a manager, I want to see incoming orders so I can manage preparation.

• As a manager, I want to change order status so customers can see progress.

• As a manager, I want to manage reservations so I can control table bookings.

• As a manager, I want to edit menu availability/prices so the customer menu stays current.

3. Functional Requirements

Authentication

FR-01: The system shall accept valid email and password; validate password length/digit; clear field errors on edit; show loading; authenticate against mock users; route customer to menu and manager to dashboard..

Menu

FR-02: The system shall load at least 15 local items; show four categories; show loading/error/retry; show daily special; disable unavailable items; support pull-to-refresh..

Search

FR-03: The system shall focus input with ref; clear and retain focus; debounce searches by 400ms; retain last five terms; show empty state..

Cart

FR-04: The system shall add, remove, increment, decrement, notes, promo apply/remove, clear cart, and live count badge..

Reservation

FR-05: The system shall hourly slots 12:00–22:00; disable unavailable slots; validate date, party size, pakistani phone format and future booking; confirm before saving; list/cancel reservations..

Orders

FR-06: The system shall create order with id/items/total/type/status/timestamp; track pending → preparing → ready → served with demo timers..

Manager Dashboard

FR-07: The system shall manager-only tab; view incoming orders; change statuses; provide reservation and menu-management areas..

FR-01: The system shall validate a syntactically valid email on submit.

FR-02: The system shall require at least eight password characters and one digit.

FR-03: The system shall authenticate against src/data/users.js after a simulated one-second delay.

FR-04: The system shall navigate a customer to Menu after successful authentication.

FR-05: The system shall navigate a manager to Dashboard after successful authentication.

FR-06: The system shall load menu mock data after a 1.5-second simulated delay.

FR-07: The system shall render menu items with keyExtractor in FlatList.

FR-08: The system shall disable Add for unavailable menu items.

FR-09: The system shall filter menu by selected category.

FR-10: The system shall apply search after 400 milliseconds of inactivity.

FR-11: The system shall show a Back to top control after 300 pixels of scrolling.

FR-12: The system shall store cart state as items, promoCode and discountPercent.

FR-13: The system shall support ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, CLEAR_CART, APPLY_PROMO and REMOVE_PROMO.

FR-14: The system shall calculate service charge at five percent.

FR-15: The system shall calculate sales tax at fifteen percent.

FR-16: The system shall create a reservation only when all required validation passes.

FR-17: The system shall create an order with Pending status when checkout is completed.

FR-18: The system shall advance demo order status after 10, 20 and 30 seconds.

FR-19: The system shall show the Manager Dashboard only when user.role is manager.

FR-20: The system shall persist orders and other local prototype data through AsyncStorage.

4. Non-Functional Requirements

• Usability: labels, validation messages, disabled states and clear navigation shall make core actions understandable on a phone.

• Performance: menu data uses FlatList; derived filtering/sorting uses useMemo; card and handlers use React.memo/useCallback; list scrolling remains responsive.

• Responsiveness: layouts use flexbox and percentage/flexible widths rather than fixed desktop dimensions.

• Maintainability: reusable components, screens, context, reducers, hooks, data and navigation are separated under src.

• Data handling: all application data is mock/local; no backend or external API is required. Persistent prototype state uses AsyncStorage.

• Reliability: loading effects and timers return cleanup functions to prevent updates after unmount.

5. Client-Side Data Model (Mock Data)

Data Set	Fields
Users	id, name, email, password, role
Categories	name
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable
Tables	id, seats
Reservations	id, date, time, partySize, table, contact, status
Orders	id, items, total, type, status, timestamp

6. UML Diagrams

The five diagrams in A1/UML model actors and use cases, domain classes and multiplicities, the customer order sequence, order lifecycle states, and the React Native component architecture. Each exported PNG is included in the repository.

01-use-case.png: Customer and Restaurant Manager interactions with ordering, reservation, account and menu-management use cases.

02-class.png: Core domain classes, attributes, operations, inheritance and associations.

03-sequence.png: Customer order flow from Add to Cart through checkout to Order Tracking.

04-state-machine.png: Order lifecycle from Pending to Preparing, Ready, Served or Cancelled.

05-component.png: React Native screens, providers, reducers and custom hooks and their dependencies.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Purpose: Collect and validate customer/manager credentials. Entry/exit: navigation stack or bottom tabs. Local data: users.js. Hooks: useState.

7.2 Menu Browsing Screen

Purpose: Load and display categorized menu items. Entry/exit: navigation stack or bottom tabs. Local data: menu.js. Hooks: useState, useEffect.

7.3 Search and Scroll Controls

Purpose: Focus/search/debounce and return-to-top behaviour. Entry/exit: navigation stack or bottom tabs. Local data: menu.js. Hooks: useRef, useState, useMemo.

7.4 Profile and Theme Screen

Purpose: Display user details and toggle theme. Entry/exit: navigation stack or bottom tabs. Local data: AuthContext, ThemeContext. Hooks: useContext, useState.

7.5 Cart Screen

Purpose: Manage quantities, notes and promo codes. Entry/exit: navigation stack or bottom tabs. Local data: CartContext, promo.js. Hooks: useReducer, useContext.

7.6 Order Summary Screen

Purpose: Calculate checkout totals and place order. Entry/exit: navigation stack or bottom tabs. Local data: CartContext, OrdersContext. Hooks: useMemo, useCallback.

7.7 Table Reservation Screen

Purpose: Validate and save table bookings. Entry/exit: navigation stack or bottom tabs. Local data: tables.js, useReservation. Hooks: custom hooks, useState.

7.8 Order Tracking and Manager Dashboard

Purpose: Track timed order status and expose manager controls. Entry/exit: navigation stack or bottom tabs. Local data: OrdersContext, AsyncStorage. Hooks: useEffect, useReducer, useContext.

8. Submission Structure

• Repository: restaurant-app-mvp

• A1/SRS.pdf

• A1/UML/ with five exported diagrams

• src/components, screens, context, reducers, hooks, data, navigation

• README.md with install steps, demo credentials, hook table, screenshots and demo video link

• Main branch with meaningful commits for Questions 1–10.